Univerzitet u Beogradu
Elektrotehnički fakultet



Analiza i optimizacija vremena pokretanja Linux sistema na RISC-V platformi Milk-V DuoS

Diplomski rad

Kandidat:
Vojin Vukčević
br. indeksa 2021/0516

Mentor:
Dr Vladimir Rajović
vanredni profesor

Beograd, 5. septembar 2026.

Sažetak

U ovom radu analizirano je i optimizovano vreme pokretanja Linux sistema na razvojnoj ploči Milk-V DuoS, zasnovanoj na čipu SG2000, pri čemu je kao glavni procesor korišćeno jezgro RISC-V arhitekture. Kao kraj postupka pokretanja uzet je trenutak u kome namenska aplikacija očita temperaturu sa senzora BME280 preko I^2C magistrale i izmerenu vrednost pošalje UDP datagramom drugom računaru, čime se potvrđuje da su svi podsistemi neophodni za rad aplikacije dovedeni u ispravno stanje.

Uspostavljen je merni postupak koji obuhvata ceo lanac pokretanja, od dovođenja napajanja do izvršavanja aplikacije. Trenutak uključenja detektuje se mikrokontrolerom, koji ujedno prosleđuje ispis sa serijske konzole host računaru, gde se granice faza izdvajaju iz vremenskih oznaka poruka koje pojedine komponente ispisuju pri pokretanju. Na osnovu tako izmerene raspodele vremena po fazama optimizacija je sprovedena fazu po fazu, od bootloadera, preko kernela, do korisničkog prostora.

Sve primenjene mere svode se na izmene konfiguracije, stabla uređaja i redosleda operacija, bez izmena u kodu aplikacije i bez promene hardvera. Ukupno vreme pokretanja smanjeno je sa 14,10 s na 3,17 s, odnosno za 77,5 %. Posmatrano isključivo nad delovima sistema koje je bilo moguće menjati, vreme je smanjeno sa 12,18 s na 1,34 s, odnosno za 89,0 %. Nakon optimizacije faze nedostupne izmeni čine većinu preostalog vremena pokretanja, čime je prostor dostupan konfiguracijom iscrpljen.

Ključne reči: namenski računarski sistemi, vreme pokretanja, RISC-V, Milk-V DuoS, Linux kernel, bootloader, stablo uređaja, korisnički prostor

Sadržaj

1 Uvod **6**

2 Pokretanje Linux sistema **7**
2.1 Definicija problema i cilj rada 7
2.2 Lanac pokretanja Linux sistema 7
2.3 Definisanje vremena pokretanja 10
2.4 Ciljna aplikacija 11

3 Merni postupak **12**
3.1 Hardverska platforma 12
3.2 Razvojno okruženje 13
3.3 Merni lanac 14
3.4 Automatizacija merenja 16
3.5 Merna nesigurnost 16
3.6 Tehnička realizacija aplikacije 17

4 Primenjene optimizacije **19**
4.1 ROM, FSBL i Sistemski firmver 19
4.2 U-Boot 20
4.3 Linux kernel 21
4.4 Korisnički prostor 23

5 Rezultati **25**
5.1 U-Boot 25
5.2 Linux kernel 26
5.3 Korisnički prostor 27
5.4 Zbirni pregled 29

6 Zaključak **32**
6.1 Tumačenje rezultata 32
6.2 Buduće izmene 32

A Firmver mernog mosta **35**

B Skripta za obradu rezultata merenja **37**

C Izvorni kod aplikacije **39**

D Dodavanje senzora BME280 **41**

E Konfiguraciona inittab datoteka **42**

Spisak slika

1 Tok pokretanja sistema na RISC-V arhitekturi. 8

2 Merna postavka. 15

3 Trajanje faza pokretanja obe konfiguracije. 30
4 Raspodela ukupnog vremena pokretanja. 30

Spisak tabela

1 Povezivanje senzora BME280 na razvojnu ploču 13
2 Verzije korišćenih komponenti 14
3 Povezivanje ploče sa mernim mostom 16
4 Poruke po kojima se prepoznaju granice faza 17
5 Mrežna konfiguracija 18

6 Raspodela vremena unutar U-Boot faze 26
7 Poređenje raspakivanja slike kernela 26
8 Raspodela vremena unutar kernel faze 27
9 Doprinos izmena korisničkog prostora 27
10 Trajanje faza pokretanja 29
11 Uštređeno vreme po fazama 29

Uvod

Namenski računarski sistemi predstavljaju računarske sisteme posebne namene, koji izvršavaju unapred određene zadatke, za razliku od računara opšte namene [1]. Projektovani su sa specifičnom funkcionalnošću, čime je ograničen spektar mogućnosti njihove primene. Pošto je ovaj tip računarskih sistema namenjen konkretnom zadatku, njegovi projektanti su u mogućnosti da optimizuju veličinu i cenu, kao i da povećaju pouzdanost i performanse. Kako složenost zadataka raste, a sa njima i očekivanja od ovakvih uređaja, raste i potreba za punim operativnim sistemom koji njima upravlja. Izbor se najčešće svodi na operativni sistem zasnovan na Linux kernelu.

Prednost Linuxa u namenskim sistemima proizlazi iz prirode njegovog modela otvorenog koda. Ovaj model omogućava da se izvorni kod prilagodi konkretnoj hardverskoj platformi i dopuni podrškom za uređaje na njoj. Uz kernel dolazi i podrška za veliki broj periferija, kao i gotovi mrežni i fajl sistemski slojevi koje projektant ne mora sam da razvija, dok širi ekosistem obezbeđuje alate i biblioteke potrebne za izgradnju sistema. Kernel je samo jedan deo sistema, pa je za dobijanje upotrebljive celine potrebno i korisničko okruženje izgrađeno posebnim alatima.

Upotreba ovakvog operativnog sistema nosi i određene nedostatke. Dok sistem bez operativnog sistema počinje da izvršava svoju funkcionalnost već nakon nekoliko desetina milisekundi, Linux sistem do potpune funkcionalnosti stiže tek nakon nekoliko sekundi, a na slabijim platformama i nakon više od deset. Iako postoje primene u kojima je to prihvatljivo, namenski sistemi se često koriste u vremenski kritičnim uslovima, u kojima uređaj mora biti spreman za rad što pre nakon uključenja. Uprkos tome, u razvoju se najčešće polazi od podrazumevanih konfiguracija, čiji je cilj olakšavanje razvoja i pouzdanost, a ne brzina pokretanja. U njima stoga po pravilu ostaje znatan prostor za skraćenje vremena pokretanja.

Predmet ovog rada je upravo skraćenje vremena od uključenja sistema do trenutka u kome on postaje spreman da izvrši svoju namensku funkciju. Kao platforma izabrana je ploča Milk-V DuoS, zasnovana na čipu SG2000, koji pored jezgra ARM arhitekture sadrži i jezgro RISC-V arhitekture. RISC-V je arhitektura koja poslednjih godina dobija sve značajniju ulogu u svetu namenskih računarskih sistema, i na ovoj ploči je predmet optimizacije upravo ona. U radu se najpre uspostavlja postupak merenja koji obuhvata sve faze pokretanja, zatim se meri referentna raspodela vremena po fazama, nakon čega se, fazu po fazu, primenjuju mere optimizacije.

Pokretanje Linux sistema

2.1 Definicija problema i cilj rada

Pokretanje Linux sistema nije jedinstven proces, već niz procesa koji se odvijaju sekvencijalno. Pod vremenom pokretanja podrazumeva se ukupno vreme potrebno da se svi ti procesi završe, odnosno vreme od dovođenja napajanja do trenutka u kome sistem počinje da obavlja svoju namensku funkciju. Potreba za skraćenjem tog vremena proizlazi iz dva osnovna razloga:

1. Poboljšanje korisničkog iskustva. Korisnik očekuje da uređaj bude spreman za upotrebu ubrzo nakon uključenja. Dugo pokretanje bez ikakve povratne informacije stvara utisak da uređaj ne radi ispravno i može ga navesti na nepotrebno ponovno pokretanje.

2. Brže uspostavljanje kritičnih funkcija. Ovaj razlog je po pravilu značajniji, jer se ovde ne gubi samo vreme, već i informacija. Ukoliko sistem obrađuje podatke od kojih zavisi bezbednost ili ispravan rad šireg sistema, svaki period u kome on još nije spreman predstavlja period u kome se ti podaci ne prikupljaju.

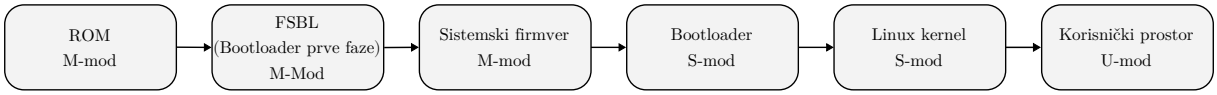
Konfiguracije koje isporučuje proizvođač ploče po pravilu nisu podešene za brzo pokretanje. Cilj proizvođača je da omogući lak početak rada na ploči, pa su podrazumevano uključeni svi uređaji prisutni na hardveru, kao i softverski paketi namenjeni razvoju i otkrivanju grešaka. Podrazumevane vrednosti pritom nisu proizvoljne, već odgovaraju profilu upotrebe koji je proizvođač imao u vidu, a koji se ne mora poklapati sa konkretnom primenom. Na konkretnom uređaju veliki deo toga nije potreban. Uz to, vreme pokretanja ne zavisi od jednog dela softvera, već je raspoređeno kroz faze koje pripadaju različitim slojevima, tako da ne postoji jedno mesto na kome se pokretanje ubrzava, nego se optimizacija radi po fazama.

Cilj ovog rada je da se utvrdi raspodela vremena pokretanja po fazama na izabranoj platformi, a zatim da se ukupno vreme skrati optimizacijom svake od tih faza. Rezultatom se smatra ukupno postignuto skraćenje vremena pokretanja, kao i doprinos svake pojedinačne faze tom skraćenju. Sve optimizacije se odnose na softverske promene, bez promene hardvera, pri čemu kod same aplikacije ostaje nepromenjen, tako da sve razlike u izmerenom vremenu potiču isključivo od izmena u konfiguraciji sistema. Potrošnja energije, veličina sistema i performanse nisu predmet optimizacije.

2.2 Lanac pokretanja Linux sistema

Pokretanje sistema je složen postupak koji se oslanja na usklađeni rad hardverskih i softverskih komponenti. Konkretna realizacija pojedinih faza razlikuje se od platforme do

platforme, ali su njihove uloge uglavnom standardizovane. U nastavku su opisane ključne faze pokretanja i zadaci koje one obavljaju. Na slici 1 prikazan je tipičan tok pokretanja sistema na RISC-V arhitekturi [2].



Slika 1: Tok pokretanja sistema na RISC-V arhitekturi.

Na slici se uz svaku fazu navodi i režim rada procesora u kome se ona izvršava, pa je pre opisa pojedinačnih faza potrebno razumeti nivoe privilegija koje definiše RISC-V arhitektura. Njihova svrha je zaštita između pojedinih slojeva softverskog steka: ukoliko se pokuša izvršavanje operacija koja u trenutnom režimu nije dozvoljena, dolazi do izuzetka, koji po pravilu dovodi do prelaza u izvršno okruženje nižeg nivoa [3]. Arhitektura definiše tri režima rada:

- Mašinski režim (M-mode) ima najviši nivo privilegija i jedini je obavezan na svakoj RISC-V hardverskoj platformi, budući da je to jedini režim sa neograničenim pristupom celokupnoj mašini.
- Nadzorni režim (S-mode) predviđen je za operativni sistem. Njegovim uvođenjem postiže se izolacija između operativnog sistema i izvršnog okruženja u kome on radi.
- Korisnički režim (U-mode) namenjen je uobičajenim korisničkim aplikacijama i štiti ostatak sistema od koda koji se u njima izvršava.

Linux sistem zahteva podršku za sva tri režima.

ROM kod je kod koji se izvršava neposredno nakon reseta ili dovođenja napajanja na ploču. U tom trenutku spoljašnja memorija još nije inicijalizovana, pa se ovaj kod smešta na sam čip. Kod je programiran unutar čipa pri proizvodnji, tako da se ne može zameniti otvorenom implementacijom i nije podložan promenama ni optimizaciji [4]. ROM kod može imati jako malo pretpostavki oko hardvera koji se nalazi van samog čipa, pa je zato jedina radna memorija koja mu je na raspolaganju statička memorija (SRAM) prisutna na samom čipu. Zadatak ROM koda jeste da iz unapred određenog izvora, najčešće fleš memorija ili multimedijska kartica (MMC), učita manji deo koda u SRAM i preda mu kontrolu. SRAM po pravilu nije dovoljno veliki da učita ceo bootloader, pa zato postoji međufaza koja se naziva FSBL (First Stage Bootloader)[1].

FSBL uspostavlja rad memorijskog kontrolera i ostalih delova sistema neophodnih za učitavanje bootloadera i sistemskog firmvera u glavnu, dinamičku memoriju (DRAM). Njegova funkcionalnost ograničena je sopstvenom veličinom. Bootloader i sistemski firmver učitava sa nekog od uređaja za skladištenje, koristeći unapred određene pomeraje od početka uređaja ili poznato ime datoteke. Interakciju sa korisnikom po pravilu ne omogućava, ali može ispisivati poruke o toku izvršavanja. FSBL je često vlasnički kod i proizvođač ga isporučuje kao binarnu datoteku. Na kraju ove faze unutar dinamičke memorije je smešten Bootloader i sistemski firmver kome FSBL predaje kontrolu [4].

[1] U literaturi se sreće i pod nazivom SPL (Secondary Program Loader)

Sistemski firmver ima ulogu da kontrolu nad sistemom preda nadzornom režimu. Za razliku od prethodnih faza, delovi sistemskog firmvera ostaju prisutni u memoriji i izvršavaju se tokom celog rada sistema. Na RISC-V arhitekturi on ujedno implementira standardizovanu spregu (SBI — Supervisor Binary Interface) između operativnog sistema i koda koji se izvršava u mašinskom režimu [5]. Preko te sprege kernel traži usluge koje zahtevaju najviši nivo privilegija, poput podešavanja sistemskog tajmera, slanja prekida drugim procesorskim jezgrima ili gašenja sistema. Time softver koji se izvršava u nadzornom režimu postaje prenosiv između različitih RISC-V implementacija.

Bootloader predstavlja sledeću fazu u pokretanju Linux sistema i ima dva osnovna zadatka: da izvrši osnovnu inicijalizaciju hardvera i da učita kernel [4]. U trenutku kada bootloader preuzme kontrolu, inicijalizovan je tek minimalni deo hardvera, pa bootloader dovodi u radno stanje samo one komponente koje su mu neophodne da učita kernel i pokrene njegovo izvršavanje. Pored kernela, bootloader može učitati i binarnu datoteku stabla (Device Tree Binary) uređaja, kao i inicijalni RAM disk, ukoliko konfiguracija sistema to zahteva. Ukoliko je slika kernela kompresovana, bootloader je pre predaje kontrole raspakuje u radnu memoriju. Pokretanje se po pravilu odvija automatski, ali bootloader najčešće omogućava i prekid tog postupka i prelazak na komandnu konzolu, što je korisno tokom razvoja. Nakon što kernel počne sa izvršavanjem, u bootloader se više ne vraća, a memoriju koju je zauzimao kernel kasnije preuzima kao slobodnu.

Podatke o prisutnom hardveru kernelu potrebno je dostaviti spolja, i tu ulogu ima stablo uređaja. Sistem je u njemu prikazan kao skup komponenti povezanih hijerarhijski, u obliku stabla. Bootloader ga učitava u memoriju i kernelu prosleđuje njegovu adresu i veličinu. Pre predaje kontrole bootloader u stablo upisuje i komandnu liniju, niz parametara kojim se kernelu zadaju podaci poput oznake uređaja sa korenskim fajl sistemom ili nivoa detaljnosti ispisa na konzoli.

Linux kernel predstavlja fazu u kojoj se sistem dovodi u stanje u kome može izvršavati korisničke programe. Kernel obavlja tri osnovna zadatka:

- upravljanje resursima — raspodeljuje procesorsko vreme, memoriju i prostor na uređajima između programa koji se istovremeno izvršavaju
- sprega prema hardveru — neposredno pristupa uređajima, preko upravljačkih programa, i obrađuje hardverske prekide
- apstrakcija prema korisničkom prostoru — programima pruža ujednačen način korišćenja resursa, tako da oni ne moraju znati na kakvom hardveru rade.

Kontrolu preuzima od bootloadera, zajedno sa adresom stabla uređaja i parametrima pokretanja. Zatim sledi rana inicijalizacija, u okviru koje se uspostavlja upravljanje memorijom, konfigurišu prekidi i pokreće raspoređivač poslova.

Na osnovu stabla uređaja kernel prepoznaje koje su komponente prisutne u sistemu i za svaku pokreće odgovarajući upravljački program. Ova faza obuhvata najveći broj pojedinačnih koraka, budući da svaki uređaj zahteva sopstvenu inicijalizaciju, koja često podrazumeva i čekanje na spremnost hardvera.

Kada su neophodni uređaji inicijalizovani, kernel montira korenski fajl sistem i pokreće prvi korisnički proces. Time se kontrola predaje korisničkom prostoru, dok kernel ostaje u memoriji i nastavlja da opslužuje zahteve programa tokom celog rada sistema.

Korisnički prostor predstavlja poslednju fazu pokretanja, u kojoj sistem postaje spreman da obavlja svoju namensku funkciju. Linux kernel sam po sebi ne čini ceo operativni sistem, već se kombinuje sa korisničkim okruženjem koje se bira prema nameni uređaja. Isti kernel se tako može upotrebiti uz GNU okruženje na računarima opšte namene, uz Android okruženje na mobilnim uređajima, ili uz kompaktna okruženja zasnovana na alatima poput BusyBoxa, kakva se koriste u namenskim sistemima. Za razliku od svih prethodnih faza, korisnički prostor se izvršava sa najnižim nivoom privilegija, pa se svaki pristup hardveru od tog trenutka odvija posredstvom kernela.

Prvi proces koji kernel pokreće naziva se init i ima identifikator 1. On je roditeljski proces svih ostalih procesa u sistemu i zadužen je za njihovo pokretanje, praćenje i, po potrebi, ponovno pokretanje. Njegova složenost se kreće od punih init sistema, koji vode računa o međusobnim zavisnostima servisa i mogu ih pokretati uporedo, do jednostavnih skriptova koji zadate programe pokreću redom.

Init zatim pokreće servise predviđene konfiguracijom sistema. Među njima su postavljanje mrežnih sprega, montiranje preostalih fajl sistema i pokretanje pozadinskih procesa. Na kraju se pokreće i sama namenska aplikacija, a trenutak u kome ona počne da obavlja svoj zadatak označava kraj postupka pokretanja.

2.3 Definisanje vremena pokretanja

Vreme pokretanja sistema nema jedinstvenu definiciju i, u zavisnosti od samog sistema, može označavati različite intervale. Za početak se uzima trenutak dovođenja napajanja. Detekcija tog trenutka zahteva spoljašnji merni uređaj, budući da sistem tada još ne postoji ni u kakvom obliku koji bi mogao započeti merenje.

Dok je početna tačka opšteprihvaćena, oko trenutka u kome se pokretanje smatra završenim postoje različita tumačenja. Načelno je to trenutak u kome je sistem uspostavljen, što može značiti više stvari: pokretanje init programa, pojavu komandne linije, ili početak izvršavanja korisničke aplikacije. Pošto je reč o namenskom sistemu, za kraj merenja logično je uzeti trenutak u kome sistem uspostavi funkcionalnost zbog koje je i projektovan.

Linux kernel uz svaku poruku ispisuje vreme proteklo od sopstvenog pokretanja, ali se ta vrednost ne može uzeti kao vreme pokretanja sistema. Ona ne obuhvata faze koje kernelu prethode, odnosno ROM kod, FSBL i sistemski firmver, kao ni rad bootloadera.

Razlikuju se i dva načina pokretanja. Hladno pokretanje (cold boot) podrazumeva pokretanje iz potpuno isključenog stanja. Toplo pokretanje (warm boot) odvija se iz stanja u kome sistem već radi, kao pri ponovnom pokretanju, pri čemu se pojedini delovi hardvera ne inicijalizuju ponovo, a deo podataka ostaje u memoriji. U ovom radu razmatra se hladno pokretanje, budući da obuhvata celokupan lanac i ima nedvosmisleno određenu nultu tačku.

Ukupno izmereni interval deli se na šest faza, koje odgovaraju koracima opisanim u odeljku 2.2: ROM kod, FSBL, sistemski firmver, bootloader, kernel i korisnički prostor. Granice između faza određuju se na osnovu poruka koje pojedine komponente ispisuju na serijskoj konzoli, pri čemu treba imati u vidu da je time početak faze pomeren za vreme koje protekne do njenog prvog ispisa.

2.4 Ciljna aplikacija

Kao referentna krajnja tačka merenja uzima se namenska aplikacija razvijena za potrebe ovog rada. Njen zadatak je da očita temperaturu sa spoljašnjeg senzora, a zatim izmerenu vrednost prosledi drugom računaru preko Ethernet veze, primenom UDP protokola (User Datagram Protocol).

Ovako postavljen zadatak predstavlja realističan primer namenskog Linux sistema. Očitavanjem senzora aplikacija prikuplja podatak iz okruženja i obrađuje ga, dok njegovo slanje preko mreže podrazumeva korišćenje mrežnog steka, odnosno upravo one funkcionalnosti koja predstavlja prednost Linux sistema nad rešenjima bez operativnog sistema. Zadatak je pritom namerno postavljen tako da obuhvata dva različita podsistema, sporu magistralu sa spoljašnjim uređajem i mrežnu spregu, pa aplikacija ne može obaviti svoj posao dok oba nisu dovedena u ispravno stanje.

Za krajnji trenutak merenja ne uzima se početak izvršavanja aplikacije, već trenutak u kome je prvi očitani podatak poslat preko mreže. Ovakav izbor je načinjen zato što se time ne potvrđuje samo da je aplikacija pokrenuta, već i da su svi podsistemi neophodni za njen rad dovedeni u ispravno stanje. Pokrenuta aplikacija koja još ne može da očita senzor ili pristupi mreži ne predstavlja spreman sistem.

Sam trenutak se detektuje na osnovu poruke koju aplikacija neposredno nakon slanja ispisuje na serijsku konzolu, čime se kraj merenja svodi na isti mehanizam kojim se prepoznaju i granice prethodnih faza. Budući da UDP ne predviđa potvrdu prijema, ispis potvrđuje da je paket predat mrežnom steku, a ne i da je stigao na odredište.

U skladu sa ograničenjima navedenim u odeljku 2.1, kod aplikacije tokom celog postupka ostaje nepromenjen, iako i njena optimizacija predstavlja jedan od mogućih pristupa skraćenju vremena pokretanja. Time se osigurava da sve razlike u izmerenom vremenu potiču isključivo od izmena u konfiguraciji sistema.

Merni postupak

3.1 Hardverska platforma

Kao platforma za realizaciju rada izabrana je razvojna ploča Milk-V DuoS. Reč je o razvojnoj ploči namenjenoj razvoju namenskih sistema, prvenstveno uređaja poput nadzornih kamera i uređaja iz oblasti interneta stvari.

Na ploči se nalazi čip SG2000 proizvođača Sophgo. Čip je heterogene građe i pored RISC-V jezgara sadrži i jezgro ARM arhitekture, kao i pomoćni mikrokontroler. Izbor jezgra koje preuzima ulogu glavnog procesora vrši se fizičkim prekidačem na samoj ploči, čime se određuje da li se sistem pokreće na RISC-V ili na ARM arhitekturi. U ovom radu koristi se RISC-V jezgro. Izbor je načinjen zato što je čip prvenstveno projektovan za rad na toj arhitekturi, a dostupna dokumentacija i podrška proizvođača znatno su obimnije za RISC-V nego za ARM stranu [6].

Ploča raspolaže sa 512 MB dinamičke memorije. Sistem se pokreće sa microSD kartice klase V10, izmerene brzine čitanja od 40 MB/s. Brzina čitanja kartice izmerena je na host računaru komandom prikazanom u isečku 1, kojom se sa uređaja čita 200 MiB podataka zaobilazeći sistemsku keš memoriju. Merenje je ponovljeno pet puta, a navedena vrednost predstavlja srednju vrednost tako dobijene serije. Ovaj podatak je značajan jer brzina čitanja neposredno utiče na vreme potrebno za učitavanje bootloadera i slike kernela.

Isečak koda 1: Merenje brzine čitanja microSD kartice.

```
1 sudo dd if=/dev/sda of=/dev/null bs=1M count=200 iflag=direct
```

Od periferija koje ploča poseduje koriste se tri. Serijska sprega sa UART-om služi za ispis poruka na konzoli i ujedno predstavlja osnovu mernog postupka opisanog u odeljku 3.3. Preko Ethernet sprege brzine 100 Mbit/s aplikacija šalje očitane vrednosti drugom računaru. Senzor je na ploču povezan preko I^2C magistrale.

Za merenje temperature upotrebljen je senzor Bosch BME280. Pored temperature, senzor meri i atmosferski pritisak i relativnu vlažnost vazduha, ali se u ovom radu koristi isključivo očitavanje temperature. Senzor je povezan na magistralu I^2C1, prema tabeli 1. Priključci CSB i SDO nemaju ulogu u prenosu podataka, već određuju režim rada i adresu uređaja: dovođenjem CSB na visok nivo bira se I^2C umesto SPI sprege, dok se dovođenjem SDO na masu adresa senzora postavlja na 0x76.

Tabela 1: Povezivanje senzora BME280 na razvojnu ploču

Pin (BME280)	Signal	Pin (DuoS)	Funkcija
1	VCC	17	napajanje senzora (3V3)
2	GND	20	masa (GND)
3	SCL	7	takt I^2C magistrale
4	SDA	11	podaci I^2C magistrale
5	CSB	17	izbor I^2C režima (3V3)
6	SDO	25	izbor adrese 0x76 (GND)

3.2 Razvojno okruženje

Izgradnja sistema vršena je na računaru sa operativnim sistemom Ubuntu 24.04, bez upotrebe virtuelnih mašina ili Docker kontejnera.

Kao osnova za izgradnju korišćen je zvanični razvojni paket (SDK - Software Development Kit), `duo-buildroot-sdk-v2` preuzet iz javnog repozitorijuma [7] . Paket objedinjuje sve komponente potrebne za izgradnju kompletne slike sistema: izvorni kod bootloadera, kernela i sistemskog firmvera, sistem za izgradnju korenskog fajl sistema, kao i cross-compiler. Struktura samog stabla nije menjana, a sve izmene tokom rada opisane su u poglavlju 4.

Izgradnja sistema se pokreće skriptom `build.sh` iz glavnog direktorijuma paketa, kojoj se kao argument prosleđuje naziv ciljne konfiguracija:

Isečak koda 2: Pokretanje izgradnje sistema.

```
1        ./build.sh milkv-duos-musl-riscv64-sd
```

Konfiguracija određuje ploču, arhitekturu, standardnu biblioteku jezika C i medijum sa koga se sistem pokreće. Rezultat postupka je jedna datoteka sa slikom sistema u formatu `.img`. Slika obuhvata sve komponente potrebne za pokretanje, odnosno bootloader, sistemski firmver, kernel i korenski fajl sistem, i na microSD karticu se upisuje alatom `dd`.

Verzije komponenti obuhvaćenih razvojnim kompletom date su u tabeli 2. Kao standardna biblioteka jezika C korišćena je musl, koja je u odnosu na uobičajenu glibc znatno manjeg obima, što je pogodnije za namenske sisteme sa ograničenim prostorom za smeštaj podataka. Prevođenje se vrši ukršteno, prevodiocem koji je sastavni deo razvojnog kompleta, budući da se izgradnja obavlja na računaru arhitekture x86-64, a ciljna arhitektura je RISC-V.

Sve izmene korenskog fajl sistema unose se preko mehanizma `BR2_ROOTFS_OVERLAY`, iz direktorijuma `./device/generic/br_overlay/musl_riscv64`. Sadržaj tog direktorijuma se pri izgradnji preslikava preko korenskog fajl sistema, čime izmene postaju sastavni deo slike. Ovakav pristup je neophodan zato što se izmene načinjene neposredno na microSD kartici gube pri svakom novom upisu slike.

Tabela 2: Verzije korišćenih komponenti

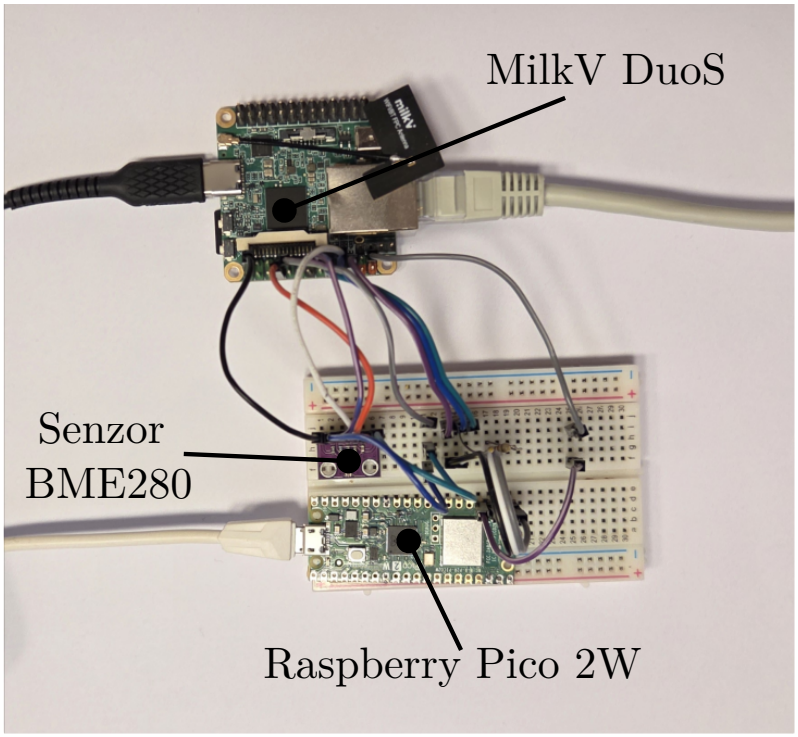
Komponenta	**Verzija**
Razvojni paket	duo-buildroot-sdk-v2
Buildroot	2025.02
U-Boot	2021.10
OpenSBI	v0.9
Linux kernel	5.10.4
Standardna biblioteka	musl
Cross-compiler	riscv64-unknown-linux-musl-gcc

3.3 Merni lanac

Merenje vremena pokretanja zasniva se na posmatranju serijske konzole sa spoljašnjeg računara. Ovakav pristup je izabran zato što kernel sopstvenim mehanizmom beleži vreme tek od trenutka svog pokretanja, pa faze koje mu prethode njime nisu obuhvaćene.

Merni lanac čine tri elementa: razvojna ploča kao predmet merenja, mikrokontrolerska ploča Raspberry Pi Pico 2W u ulozi mosta između serijske sprege i računara, i host računar na kome se zapis beleži i obrađuje. Umesto uobičajenog pretvarača iz serijske sprege u USB, upotrebljen je mikrokontroler zato što pretvarač ne može da registruje trenutak dovođenja napajanja, pa bi merenje počinjalo tek od prvog primljenog znaka, čime bi prva faza pokretanja ostala neobuhvaćena.

Pico je sa razvojnom pločom povezan preko serijske sprege, prema tabeli 3, pri čemu su obe ploče napajane nezavisno i povezane zajedničkom masom. Pored prenosa podataka, Pico obavlja i detekciju trenutka uključenja: njegov ulaz GPIO18 povezan je na naponsku liniju od 3,3 V razvojne ploče, a prelazak tog ulaza u visoko stanje prekidnom rutinom označava početak merenja. Trenutak dolaska prvog znaka sa serijske linije beleži se sa istog brojača, a razlika između ta dva trenutka predstavlja trajanje prve faze pokretanja, odnosno vreme od dovođenja napajanja do prvog ispisa ROM koda. Celokupna merna postavka prikazana je na slici 2.



Slika 2: Merna postavka.

Firmver mernog mosta realizovan je u MicroPython okruženju, a njegov potpuni izvorni kod dat je u Dodatku A. Zadatak firmvera je trostruk: da zabeleži trenutak uključenja razvojne ploče, da zabeleži trenutak dolaska prvog znaka sa njene serijske linije, i da izmerenu razliku, zajedno sa svim primljenim znacima, prosledi host računaru.

Ulaz GPIO18 konfigurisan je sa unutrašnjim otpornikom prema masi, a prekid je postavljen na obe ivice signala. Pri rastućoj ivici prekidna rutina upisuje vrednost brojača, dok se pri opadajućoj sve oznake brišu, čime se merni most po isključenju ploče sam vraća u početno stanje i naredno merenje ne zahteva nikakvu radnju nad njim.

Dolazak prvog znaka ne detektuje se prekidom, već proverom prijemnog bafera u glavnoj petlji, pri čemu se obe vremenske oznake uzimaju sa istog brojača. Brojač sa koga se uzimaju obe oznake meri u mikrosekundama, ali merenje unosi dva kašnjenja: znak se u baferu pojavljuje tek pošto je u celosti primljen, a uočava se tek pri narednom prolazu kroz petlju. Oba su ispod jedne milisekunde, pa su zanemarljiva u odnosu na trajanja koja se mere.

Izmerena razlika se ispisuje kao poseban red na isti izlaz kojim se prosleđuju i znaci sa razvojne ploče, pa se u zapisu pojavljuje među njenim porukama i prepoznaje se na isti način kao i ostale granice faza. Prenos je dvosmeran, pa se znaci uneti na host računaru prosleđuju ploči, čime je omogućen rad sa komandnom konzolom bootloadera.

Na host računaru se zapis beleži programom `grabserial`, koji uz svaki primljeni red dodaje vremensku oznaku:

Isečak koda 3: Pokretanje programa grabserial.

```
grabserial -d /dev/ttyACM0 -b 115200 -t -e 20 -o boot_full.log
```

Prekidač -t uz svaki red dodaje vreme proteklo od pokretanja programa, izraženo u sekundama. Pošto se trajanja pojedinih faza računaju kao razlike vremenskih oznaka, izbor nulte tačke tog brojanja ne utiče na rezultat. Prekidačem -e merenje se prekida

po isteku dvadeset sekundi, što je dovoljno da obuhvati i najduže pokretanje referentne konfiguracije. Zapis se čuva u tekstualnoj datoteci, koja se potom obrađuje postupkom opisanim u odeljku 3.4.

Uključenje se vrši ručno, priključivanjem naponskog kabla. Između uzastopnih merenja ostavlja se pauza od nekoliko sekundi, kako bi napajanje ploče u potpunosti opalo i svako merenje obuhvatilo pokretanje iz isključenog stanja.

Tabela 3: Povezivanje ploče sa mernim mostom

Pin (DuoS)	Signal	Pin (Pico)
8	TX	GP17 (RX)
10	RX	GP16 (TX)
6	GND	GND
1	3V3	GP18 (detekcija napajanja)

3.4 Automatizacija merenja

Zapis koji `grabserial` proizvodi sadrži nekoliko stotina redova, uz vremensku oznaku pored svakog. Ručno izdvajanje granica faza iz takvog zapisa bilo bi sporo i podložno greškama, pa je obrada automatizovana `bash` skriptom.

Skripta u zapisu pronalazi redove koji označavaju početak pojedinih faza i iz njih izdvaja vremensku oznaku. Prepoznavanje se vrši na osnovu karakterističnih poruka koje pojedine komponente ispisuju pri pokretanju, datih u tabeli 4. Trajanje svake faze računa se kao razlika između vremenske oznake njenog početka i početka naredne faze, dok se trajanje poslednje faze računa do trenutka u kome aplikacija ispiše potvrdu o poslatom očitavanju. Potpuni izvorni kod skripte dat je Dodatku B.

Vreme prve faze ne izračunava se iz zapisa, već se preuzima iz reda koji ispisuje Pico mikrokontroler, budući da se ono odnosi na period pre prvog ispisa na serijskoj liniji.

Rezultat obrade dopisuje se u zbirnu datoteku, uz redni broj merenja, čime se dobija zapis cele serije. Jedno merenje obuhvata prekid i ponovno uspostavljanje napajanja, čekanje da se pokretanje završi, i pokretanje skripte nad novonastalim zapisom. Postupak je za svaku konfiguraciju ponovljen 50 puta.

3.5 Merna nesigurnost

Vreme pokretanja nije potpuno determinisana veličina, pa se pri uzastopnim pokretanjima iste konfiguracije dobijaju međusobno različite vrednosti. Iz tog razloga se svaka konfiguracija meri više puta, a rezultat izražava srednjom vrednošću serije. Referentna konfiguracija i konačna, potpuno optimizovana konfiguracija merene su po 50 puta, budući da se njihovo poređenje uzima kao glavni rezultat rada. Međukonfiguracije, koje služe za utvrđivanje doprinosa pojedinačnih mera, merene su manji broj puta, budući da im je svrha procena reda veličine dobitka, a ne rezultat rada. Broj merenja iz kojih potiče svaka vrednost naveden je uz odgovarajuću tabelu.

Pored slučajnog rasipanja, koje se povećanjem broja merenja smanjuje, postoji i odstupanje koje ostaje nepromenjeno bez obzira na broj ponavljanja. Ono proizlazi iz načina

Tabela 4: Poruke po kojima se prepoznaju granice faza

Faza	**Poruka**
ROM	ROM phase
FSBL	SCS/
Sistemski firmver	OpenSBI v0.9
Bootloader	U-Boot 2021.10
Kernel	Starting kernel...
Korisnički prostor	Run /sbin/init
Kraj merenja	Temperature sent

na koji se granice faza određuju: vremenska oznaka označava trenutak u kome je poruka ispisana, a ne trenutak u kome je posao koji joj prethodi završen, pa je početak svake faze pomeren za vreme koje protekne do njenog prvog ispisa.

Uz to, i sam prenos poruke na serijsku konzolu traje. Pri brzini od 115200 bita u sekundi jedan red ispisa košta približno 6 ms, pa broj ispisanih redova neposredno utiče na izmereno trajanje faze. Ovo odstupanje nije zanemarljivo: referentna konfiguracija radi sa punim ispisom na konzoli, a optimizovana sa utišanim, pa jedan deo razlike u izmerenom vremenu potiče od količine ispisa, a ne od rada koji sistem obavlja.

3.6 Tehnička realizacija aplikacije

Aplikacija je realizovana u programskom jeziku C i prevedena prevodiocem koji je sastavni deo razvojnog paketa. Potpuni izvorni kod dat je u Dodatku C.

Očitavanje senzora oslanja se na postojeći drajver iz stabla kernela, koji senzor prijavljuje kroz podsistem za industrijske ulazno-izlazne uređaje. Da bi ga kernel prepoznao, u stablo uređaja je dodat čvor koji opisuje senzor na magistrali I²C1, sa adresom 0x76, dat u Dodatku D, dok su u konfiguraciju kernela uključene opcije `CONFIG_IIO` i `CONFIG_BMP280`.

Sam čvor u stablu uređaja nije dovoljan. Pinovi preko kojih je senzor povezan u podrazumevanoj konfiguraciji ploče nisu postavljeni na funkciju I²C magistrale, pa kernel pri pokušaju pridruživanja upravljačkog programa ne dobija odgovor sa magistrale. Njihova funkcija se zato mora postaviti pre nego što kernel počne da prepoznaje uređaje, dakle u bootloaderu.

Time se povezivanje senzora obavlja tokom inicijalizacije kernela, pre nego što korisnički prostor uopšte započne rad, pa aplikacija po pokretanju zatiče senzor spreman za očitavanje. Vrednost se čita iz datoteke koju drajver izlaže u sysfs hijerarhiji, u tekstualnom obliku i izražena u milistepenima Celzijusa.

Očitana vrednost se oblikuje u tekstualnu poruku i šalje drugom računaru datagramom, preko utičnice tipa `SOCK_DGRAM` [8]. Obe strane koriste statički dodeljene adrese, prema tabeli 5, a veza je ostvarena neposredno, bez mrežnog prekidača. Time se iz postupka merenja uklanjaju kašnjenja koja bi unosila mrežna oprema, kao i vreme potrebno za pribavljanje adrese sa servera.

Neposredno nakon slanja aplikacija ispisuje potvrdnu poruku na serijsku konzolu, čime se određuje kraj merenog intervala. U referentnoj konfiguraciji aplikacija se pokreće iz

skripte `S99user`, koju `init` sistem izvršava pri kraju postupka pokretanja. Izmene načina pokretanja predstavljaju jednu od primenjenih mera optimizacije i opisane su u odeljku 4.4.

Tabela 5: Mrežna konfiguracija

Strana	**Sprega**	**Adresa**
Host računar	eno1	192.168.10.1/24
Razvojna ploča	eth0	192.168.10.2/24

Primenjene optimizacije

Optimizacija je sprovedena po fazama pokretanja, redom kojim se one izvršavaju: od bootloadera, preko kernela, do korisničkog prostora. Ovakav redosled je izabran zato što svaka faza priprema okruženje u kome se naredna izvršava, pa se izmene u kasnijim fazama pouzdanije ocenjuju kada su prethodne već dovedene u konačno stanje.

Podela na faze zadržana je i kao podela posla, zato što svaka od njih predstavlja zaseban softverski sloj, sa sopstvenim načinom konfigurisanja i sopstvenim alatima [9], [10]. Ne postoji jedinstveno mesto na kome se pokretanje ubrzava, pa se ni optimizacija ne može sprovesti jedinstvenim postupkom. Razlikuje se i način na koji se utvrđuje gde se vreme troši: dok se kod bootloadera izvršava sekvencijalno, pa je razmak između dva ispisa neposredna mera trajanja onoga što se između njih odvija, kernel koristi niti i odloženo pridruživanje upravljačkih programa, pa se trošak pojedine operacije ne mora pojaviti na mestu na kome je ona pokrenuta.

Unutar svake faze primenjen je isti postupak. Najpre se iz zapisa pokretanja utvrđuje gde se vreme troši, zatim se ispituje uzrok utvrđenog troška, pa se primenjuje odgovarajuća izmena. Nakon ponovne izgradnje slike sistema vrši se merenje i provera ispravnosti rada. Ukoliko izmena ne donese očekivano skraćenje ili naruši rad sistema, ona se poništava i postupak se ponavlja sa drugačijom pretpostavkom. Polazna tačka svake izmene je, dakle, merenje, a ne pretpostavka o tome gde bi vreme moglo da se troši.

Većina primenjenih mera svodi se na uklanjanje rada koji na ovoj platformi nije potreban i nema drugih posledica po sistem. Izuzetak je uklanjanje kompresije slike kernela, kojim se skraćuje vreme pokretanja, ali se veličina slike višestruko povećava. Takav kompromis je prihvatljiv budući da veličina slike, u skladu sa ograničenjima navedenim u odeljku 2.1, nije predmet optimizacije u ovom radu.

Poglavlje je podeljeno prema fazama pokretanja. Odeljak 4.1 obrazlaže zašto prve tri faze nisu bile predmet izmena, dok odeljci 4.2 do 4.4 opisuju izmene primenjene na bootloader, kernel i korisnički prostor.

4.1 ROM, FSBL i Sistemski firmver

Prve tri faze pokretanja nisu bile predmet optimizacije u ovom radu, ali iz različitih razloga.

ROM kod je pri proizvodnji upisan u samu integrisanu strukturu čipa i ne može se izmeniti niti zameniti. Njegov udeo u ukupnom vremenu pokretanja pritom iznosi oko 17 ms, pa on ni ne predstavlja smislenu metu.

FSBL je, za razliku od ROM koda, dostupan u izvornom obliku u okviru razvojnog kompleta. Pritom nije standardna komponenta: iako po ulozi odgovara prvom stepenu bootloadera, ne pripada nijednom od uobičajenih rešenja poput U-Boot SPL-a, već predstavlja rešenje proizvođača za koje ne postoji zasebna dokumentacija. Izmene u toj fazi

zahtevale bi zadiranje u inicijalizaciju dinamičke memorije, podešavanje taktova i raspodelu memorijskog prostora, dakle u nivo na kome se konfiguracijom ne može upravljati.

Sistemski firmver je otvorena komponenta sa javnom dokumentacijom, pa razlog za njegovo izostavljanje nije u dostupnosti. Njegovo trajanje iznosi nešto manje od 170 ms i u velikoj meri se svodi na poslove koji se moraju obaviti, poput postavljanja rukovalaca zamkama i pripreme za prelazak u nadzorni režim.

Ovakva odluka utiče i na tumačenje rezultata. Kako se ukupno vreme pokretanja smanjuje, udeo neoptimizovanih faza raste, pa one nakon sprovedenih izmena čine većinu preostalog vremena pokretanja.

4.2 U-Boot

Analiza statusnih poruka koje bootloader ispisuje tokom rada pokazala je da se najveći deo vremena u ovoj fazi troši na svega tri stavke. Sve preostale operacije, poput premeštanja bootloadera u memoriju, prijave uređaja, učitavanja okruženja i podešavanja konzole, zajedno traju znatno kraće.

Podešavanje režima rada SD kartice

Najveći pojedinačni trošak predstavljalo je podešavanje vremenskog usklađivanja pri pristupu SD kartici. Kartica je radila u režimu UHS SDR104, na taktu od 208 MHz. Na toj brzini podatak je na liniji ispravan tokom vrlo kratkog dela svakog takta, pa kontroler mora unapred utvrditi u kom trenutku da ga očita. Postupak se sastoji od ispitivanja svih mogućih pomeraja i izbora sredine opsega koji radi ispravno.

Pregled izvornog koda upravljačkog programa pokazao je da se svaki od 128 pomeraja proverava do 51 put, čime se dobija do 6528 pojedinačnih naredbi na magistrali. Time je ujedno objašnjena i izrazito mala rasejanost tog dela merenja, budući da je reč o unapred određenom broju prenosa bez čekanja na spoljašnje događaje.

Razmatrana su dva rešenja. Prvo je smanjenje broja ponavljanja po pomeraju, čime se zadržava puna brzina čitanja, ali se povećava rizik da u prihvaćeni opseg uđe pomeraj koji radi tek povremeno, što bi se ispoljilo kao neispravno čitanje pod izmenjenim uslovima rada. Drugo je prelazak na sporiji režim rada, kod koga je vremenski prozor dovoljno širok da usklađivanje uopšte nije potrebno.

Izabrano je drugo rešenje. Predmet ovog rada je vreme pokretanja, a ne propusna moć, dok je odnos dobitka i gubitka izrazito jednostran: usklađivanje traje više od dve sekunde, a jedino što se njime dobija jeste nešto brže čitanje jedne datoteke od tri megabajta.

Izmena je sprovedena maskiranjem odgovarajućih bitova u polju sposobnosti kontrolera, kroz svojstvo `sdhci-caps-mask` u stablu uređaja. Time se u režim brzog rada ne ulazi, pa usklađivanje izostaje. Vrednost maske je naknadno proširena zbog ponašanja kernela, što je opisano u odeljku 4.3.

Kompresija slike kernela

Slika kernela je u referentnoj konfiguraciji kompresovana postupkom LZMA i raspakuje je bootloader pre predaje kontrole. Poređenjem više postupaka kompresije utvrđeno je da vreme raspakivanja znatno prevazilazi vreme koje se uštedi na čitanju manje datoteke sa kartice.

Konfiguracija je stoga izmenjena tako da se slika kernela ne kompresuje. Time slika raste sa oko tri na oko sedam megabajta, uz odgovarajuće produženje čitanja sa kartice, ali se u potpunosti uklanja vreme raspakivanja. Rezultati poređenja dati su u odeljku 5.1.

Ova izmena i prethodna se međusobno uslovljavaju. Prelazak na sporiji režim rada kartice produžava čitanje, a nekompresovana slika povećava količinu podataka koju treba pročitati, pa se deo dobitka iz jedne izmene vraća kroz drugu. Uprkos tome, obe zajedno donose znatno skraćenje, jer su uklonjena vremena višestruko veća od dodatnog vremena čitanja.

Kao posledica ove izmene, iz konfiguracije bootloadera uklonjena je i podrška za postupke dekompresije LZMA i LZ4, koja time postaje nepotrebna.

Uklanjanje nepotrebnih komponenti

Iz konfiguracije bootloadera uklonjene su komponente koje na ovoj platformi ne učestvuju u pokretanju: podrška za prikaz na ekranu i prateća podrška za slike u BMP formatu, vendorska komanda za upravljanje video izlazom, kao i generisanje nasumične hardverske adrese mrežne sprege, budući da se mreža u ovoj fazi ne koristi.

Uklanjanje čekanja pri automatskom pokretanju

Bootloader pre pokretanja jezgra ostavlja korisniku vreme da prekine postupak i pređe na komandnu konzolu. Ta mogućnost je korisna tokom razvoja, ali u konačnoj konfiguraciji uređaja predstavlja čisto čekanje. Vrednost `bootdelay` je stoga postavljena na nulu, čime se pokretanje nastavlja bez zastoja.

Izmena je namerno primenjena poslednja, jer se do tada pristup konzoli bootloadera koristio pri ispitivanju ostalih izmena.

Priprema magistrale za senzor

Ova izmena, za razliku od ostalih u ovoj fazi, ne skraćuje vreme pokretanja. Pinovi na koje je senzor povezan podrazumevano rade kao obični ulazno-izlazni pinovi, pa senzor nije dostupan sve dok im se ne postavi funkcija I^2C magistrale. To mora biti urađeno pre nego što kernel počne da prepoznaje uređaje, dakle u bootloaderu.

4.3 Linux kernel

Za utvrđivanje raspodele vremena unutar ove faze najpre je isprobana mogućnost koju pruža sam kernel. Zadavanjem parametra `initcall_debug` kernel uz svaku inicijalizacionu rutinu ispisuje poruku pre njenog poziva i posle povratka iz nje, zajedno sa izmerenim trajanjem, čime se dobija pregled troška svakog pojedinačnog upravljačkog programa i podsistema.

Pokazalo se, međutim, da taj pregled ne obuhvata mesto na kome se vreme zaista troši. Ukupno trajanje svih inicijalizacionih rutina iznosilo je svega nekoliko stotina milisekundi, dok je najveći deo faze otpadao na čekanje koje se odvija van njih, pri montiranju korenskog fajl sistema. Iz tog razloga je i u ovoj fazi primenjen postupak korišćen kod bootloadera, zasnovan na razmacima između vremenskih oznaka poruka ispisanih na konzoli.

Ispis na konzoli

Prvo je utvrđeno da znatan deo izmerenog trajanja ne potiče od rada kernela, već od prenosa njegovih poruka na serijsku liniju. Nivo detaljnosti ispisa određuje se parametrom `loglevel`, koji se kernelu prosleđuje pri pokretanju i uzima vrednosti od 0 do 8, pri čemu veća vrednost propušta na konzolu i poruke nižeg značaja. Kernel razlikuje osam nivoa značaja, pa vrednost 8 propušta sve poruke, a veće vrednosti nemaju dodatnog dejstva. U referentnoj konfiguraciji proizvođač zadaje vrednost 9, koja je stoga istovetna sa 8, pa ispis nije ograničen i tokom pokretanja se na konzolu prosledi nekoliko stotina redova. Pri brzini od 115200 bita u sekundi prenos jednog reda traje oko šest milisekundi, čime samo slanje poruka postaje jedna od najskupljih stavki u fazi.

Parametar je stoga postavljen na vrednost 0, čime na konzolu prolaze još samo poruke o otkazu sistema. Poruke se pri tome i dalje beleže u prstenastom baferu u memoriji, odakle se nakon pokretanja mogu pročitati bez ikakvog troška na serijskoj liniji. Da bi ceo zapis pokretanja stao u taj bafer, njegova veličina je povećana.

Izmena je u konačnoj konfiguraciji primenjena poslednja, budući da su poruke kernela tokom rada na ostalim izmenama bile neophodne za praćenje njihovog dejstva.

Podešavanje režima rada SD kartice

Nakon tumačenja poruka na konzoli pokazalo se da najveći deo vremena otpada na čekanje na spremnost SD kartice pre montiranja korenskog fajl sistema. Poređenjem sa merenjima iz prethodne faze utvrđeno je da je reč o istom postupku usklađivanja opisanom u odeljku 4.2, koji se u kernelu ponovo sprovodi uprkos izmeni koja ga je u bootloaderu uklonila.

Uzrok je u tome što bootloader i kernel na isti podatak reaguju različito. U najbržem režimu rada usklađivanje je obavezno i oba ga sprovode. U sporijem režimu nije obavezno, ali kontroler i dalje javlja da ga želi. Bootloader tu informaciju ne čita i usklađivanje preskače, dok je kernel čita i postupa po njoj.

Maska iz prethodne faze je zato proširena, kao u kodu 4, tako da se uklanja i ta oznaka, čime usklađivanje izostaje u oba slučaja.

Isečak koda 4: Postavljanje maske.

```
1         sdhci-caps-mask = <0x2006 0x0>;
```

Vrednost 0x2006 objedinjuje tri bita: `0x0002` i `0x0004`, kojima se uklanja podrška za režime SDR104 i DDR50 i koji potiču iz izmene opisane u odeljku 4.2, i `0x2000`, dodat u ovoj fazi, kojim se uklanja zahtev za usklađivanjem u režimu SDR50. Pošto se radi o istom stablu uređaja koje čita i bootloader, izmena se odražava na obe faze, ali bez posledica po bootloader, budući da on taj poslednji bit ionako ne čita.

Uklanjanje nepotrebnih komponenti

Iz konfiguracije kernela uklonjena je podrška za uređaje i podsisteme koji se na ovoj platformi ne koriste: Bluetooth, bežična mreža zajedno sa pripadajućim upravljačkim programom, zvučni podsistem, podrška za USB kamere, podrška za SPI NOR fleš memoriju koja na ploči ne postoji, kao i mehanizam obustave rada. Istovremeno su uključene opcije neophodne za rad senzora.

Ovakve izmene ne uklanjaju čekanje na hardver, već smanjuju veličinu slike jezgra, čime se skraćuje vreme njenog čitanja sa kartice.

4.4 Korisnički prostor

Za utvrđivanje raspodele vremena unutar ove faze bilo je potrebno dopuniti postojeći ispis. Za razliku od bootloadera i kernela, koji tokom rada ispisuju poruke same od sebe, skripte korisničkog prostora izvršavaju se tiho. U petlju koja pokreće pojedinačne skripte dodat je zato ispis njihovog naziva, čime svaka od njih dobija vremensku oznaku u zapisu i njihov pojedinačni trošak postaje merljiv.

Kriterijum za odlučivanje šta se uklanja proizlazi iz zahteva same aplikacije. Njoj su potrebne tačno četiri stvari: postavljena funkcija pinova I^2C magistrale, pridružen upravljački program senzora, dodeljena adresa na mrežnoj sprezi i uspostavljena veza na fizičkom sloju. Svaka druga operacija koja se izvrši pre nje predstavlja čisto kašnjenje.

Uspostavljanje mrežne veze

Merenje je pokazalo da uspostavljanje veze na mrežnoj sprezi traje oko sekundu. Uzrok nije u samom podizanju sprege, koje se završava gotovo trenutno, već u postupku kojim dve strane usaglašavaju brzinu i način prenosa. Tokom tog vremena procesor ne radi ništa, već se čeka na završetak razmene.

Primenjeno rešenje se sastoji od dve izmene. Podizanje sprege i dodela adrese premešteni su u konfiguracionu datoteku `inittab`, `/etc/inittab`, datu u Dodatku E, čime počinju što ranije i dalje se odvijaju uporedo sa ostatkom pokretanja. Uz to je brzina i način prenosa zadat neposredno na obe strane veze, čime pregovaranje postaje nepotrebno.

Uklanjanje nepotrebnih skripti

Pregledom pojedinačnih troškova utvrđeno je da se veći deo skripti koje se izvršavaju pri pokretanju odnosi na usluge koje ovaj sistem ne koristi. Uklonjene skripte mogu se podeliti u tri grupe.

Prvu čine mrežne usluge. Uklonjeno je pribavljanje adrese sa servera, budući da se adresa u ovom sistemu zadaje unapred. Uklonjeno je i podešavanje mrežnih sprega, jer je jedina sprega navedena u konfiguraciji povratna. Najskuplja među ovim skriptama bila je upravo ona koja pribavlja adresu sa servera.

Drugu grupu čine pozadinski procesi koji tokom rada ostaju aktivni. To su beleženje sistemskih poruka, izvršavanje zakazanih poslova i udaljeni pristup sa shellom. Svi ostaju pokrenuti do gašenja sistema, ali nijedan nije potreban aplikaciji, a zapisi koje bi prvi od njih čuvao dostupni su i preko prstenastog bafera u memoriji.

Treću grupu čini podrška za hardver koji se ne koristi, u ovom slučaju Bluetooth.

Vredi napomenuti da se trošak skripte ne uklanja pražnjenjem njenog sadržaja, budući da svako njeno pokretanje podrazumeva stvaranje novog procesa bez obzira na to šta se u njemu izvršava. Skripte su zato preimenovane tako da ih mehanizam koji ih pokreće više ne prepoznaje.

Uklanjanje multimedijalnog podsistema

Skripta `loadsystemko.sh`, koju izvršava poslednja stavka referentne konfiguracije, učitavala je osamnaest upravljačkih programa vezanih za multimedijalne mogućnosti čipa: prihvat slike sa kamere, obradu i prikaz video zapisa, kodovanje. Nijedna od tih mogućnosti se u ovom radu ne koristi, pa su ti programi izostavljeni iz učitavanja. Uz sam rad koji

ti programi obavljaju, svako od osamnaest učitavanja podrazumeva i stvaranje novog procesa i čitanje datoteke sa kartice.

Zadržan je jedino program za termičko upravljanje, koji pri porastu temperature snižava radni takt procesora. Reč je o mehanizmu zaštite hardvera, dok je njegov doprinos vremenu pokretanja zanemarljiv, pa njegovo uklanjanje ne bi bilo opravdano.

Skraćivanje lanca pokretanja

Nakon uklanjanja nepotrebnih skripti, mehanizam koji ih pokreće ostao je u sistemu radi izvršavanja jedne jedine stavke. Aplikacija se stoga pokreće neposredno iz konfiguracione datoteke init programa, kako je prikazano u Dodatku E, čime se uklanjaju posrednici i pripadajući troškovi stvaranja procesa.

Rezultati

Prethodna poglavlja opisala su pojedinačne izmene primenjene na svaku fazu procesa pokretanja sistema. Ovo poglavlje objedinjuje njihov efekat kroz merenja izvršena nad dve potpuno funkcionalne konfiguracije: referentnom, koja odgovara neizmenjenom paketu proizvođača, i optimizovanom, koja sadrži sve opisane izmene. Obe konfiguracije izvršavaju identičan zadatak, očitavanje temperature sa BME280 senzora preko I^2C magistrale i slanje vrednosti UDP datagramom ka host računaru, pa su njihova vremena neposredno uporediva. Svaka konfiguracija merena je 50 puta, a sve navedene vrednosti predstavljaju srednje vrednosti tako dobijenih serija.

Ukupno vreme pokretanja sistema smanjeno je sa 14,10 ± 0,01 s na 3,171 ± 0,003 s, što predstavlja smanjenje od 77,5 %, odnosno 4,44 puta brže pokretanje. Ova brojka, međutim, potcenjuje domet primenjenih izmena, jer u sebi sadrži i faze koje nisu bile predmet optimizacije. Faze ROM, FSBL i OpenSBI čine kod proizvođača nad kojim korisnik nema kontrolu i zauzimaju 1,92 ± 0,01 s u referentnoj, odnosno 1,834 ± 0,001 s u optimizovanoj konfiguraciji. Razlika od oko 80 ms potiče gotovo u celosti od faze FSBL, koja nije bila predmet izmena. Ona je posledica toga što u referentnoj seriji deo merenja te faze odudara naviše, čime je srednja vrednost povišena, dok u optimizovanoj seriji takvih merenja nema. Pojava je opisana u odeljku 5.4. Posmatrano isključivo nad delom koji je bilo moguće menjati, dakle nad U-Bootom, kernelom i korisničkim prostorom, vreme je smanjeno sa 12,179 ± 0,006 s na 1,338 ± 0,003 s, odnosno za 89,0 % (faktor 9,10).

Odeljci koji slede prikazuju rezultate po fazama, redom kojim su izmene opisane u poglavlju 4, dok su u odeljku 5.4 dati zbirni pregled i poređenje doprinosa pojedinih faza.

5.1 U-Boot

Faza U-Boot obuhvata period od predaje kontrole iz OpenSBI-ja do trenutka kada U-Boot skoči na ulaznu tačku kernela. U referentnoj konfiguraciji ova faza traje 4,918 ± 0,002 s i predstavlja najveći pojedinačni blok celokupnog procesa pokretanja, sa udelom od 34,9 % ukupnog vremena. U optimizovanoj konfiguraciji svedena je na 533,0 ± 0,7 ms, čime je ostvarena ušteda od 4,385 ± 0,002 s, odnosno smanjenje od 89,2 %. Ta ušteda čini 40,1 % ukupno uštedjenog vremena, više nego bilo koja druga faza.

Analiza vremenskih razmaka između uzastopnih ispisa U-Boota pokazala je da se gotovo ceo trošak faze svodi na tri stavke, prikazane u tabeli 6. Postupak podešavanja SD kartice, opisan u odeljku 4.2, u potpunosti je uklonjen prelaskom na sporiji režim rada. Podešavanje SD kartice izmereno je kao razmak između vremenskih oznaka dva uzastopna ispisa U-Boota, kao srednja vrednost pet merenja. Trajanje dekompresije preuzeto je iz poruke koju U-Boot sam ispisuje po raspakivanju slike, takođe kao srednja vrednost pet merenja. Vrednost bootdelay nije merena, već je zadata konfiguracijom, dok je stavka ostalo dobijena oduzimanjem prve tri stavke od ukupnog trajanja faze.

Tabela 6: Raspodela vremena unutar U-Boot faze

Stavka	Trajanje	Udeo u fazi
Podešavanje SD	2301,5 ms	46,8 %
Dekompresija slike	1178 ms	24,0 %
`bootdelay`	1000 ms	20,3 %
Ostalo	438,5 ms	8,9 %

Tabela 7: Poređenje raspakivanja slike kernela

Postupak	Vreme raspakovanja
LZMA	1178 ms
GZIP	199 ms
LZ4	220 ms
Bez kompresije	5 ms

Za dekompresiju slike kernela izvršeno je posebno merenje, u kome su međusobno upoređena sva četiri raspoloživa izbora. Rezultati su dati u tabeli 7. Za svaki postupak izvršeno je pet merenja, a navedene vrednosti predstavljaju srednje vrednosti tako dobijenih serija. I ovde su vrednosti preuzete iz poruke koju U-Boot ispisuje po raspakivanju, pa ne zavise od prenosa na serijsku konzolu.

LZMA je najlošiji od svih raspoloživih izbora na ovom hardveru, i to sa razlikom od gotovo pune sekunde u odnosu na naredni. Izabrana je nekompresovana slika, čime je ostvarena ušteda od približno 1173 ms. Preostalih 5 ms nije nula jer i dalje postoji kopiranje 7,7 MiB sa adrese učitavanja na adresu izvršavanja.

Postavljanjem vrednosti bootdelay na nulu uklonjena je puna sekunda čekanja.

Smanjenjem konfiguracije U-Boota binarna datoteka smanjena je sa 528 303 na 459 679 bajtova, odnosno za 13 %.

5.2 Linux kernel

Faza kernela obuhvata period od skoka na ulaznu tačku kernela do predaje kontrole prvom korisničkom procesu. U referentnoj konfiguraciji traje 4,336 ± 0,005 s, odnosno 30,8 % ukupnog vremena pokretanja. U optimizovanoj konfiguraciji svedena je na 710,1 ± 3,3 ms, što predstavlja uštedu od 3,626 ± 0,006 s i smanjenje od 83,6 %. Ta ušteda čini 33,2 % ukupno uštedjenog vremena.

Znatan deo izmerenog trajanja u referentnoj konfiguraciji otpada na prenos poruka na serijsku konzolu. Pri brzini od 115200 bita u sekundi jedna linija ispisa košta približno 6 ms, a tokom pokretanja se ispiše nekoliko stotina linija. Postavljanjem parametra loglevel na vrednost 0 uklonjeno je oko 1,44 s.

Koliko je taj trošak izražen pokazuje i pokušaj merenja sa parametrom `initcall_debug`. Pri punom nivou ispisa on je tokom faze kernela proizveo 898 linija, što pri 6 ms po liniji iznosi oko 5,4 s. Merenje je prijavilo ukupno trajanje od 5,55 s, dakle gotovo isključivo trošak sopstvenog ispisa.

Nakon utišavanja konzole postalo je vidljivo gde se preostalo vreme troši. Tabela 8 prikazuje raspodelu izmerenu nad konfiguracijom sa već primenjenim izmenama u fazi U-Boot, u kojoj faza kernela traje 2872,67 ms.

Tabela 8: Raspodela vremena unutar kernel faze

Stavka	**Trajanje**	**Udeo u fazi**
Inicijalizacija	287,70 ms	10 %
Podešavanje SD	2297 ms	80 %
Montiranje fajl sistema	287,97 ms	10 %

Vrednosti u tabeli 8 potiču iz jednog zapisa pokretanja, a ne iz serije merenja, budući da im je svrha bila da pokažu raspodelu vremena unutar faze, a ne da posluže kao rezultat.

Čekanje na spremnost kartice pokazalo se kao isti postupak usklađivanja koji je u fazi U-Boot već bio uklonjen, ali koji kernel i dalje sprovodi iz razloga opisanih u odeljku 4.3. Proširenjem maske u stablu uređaja taj postupak je uklonjen i iz ove faze, čime je ostvarena ušteda od oko 2,3 s.

5.3 Korisnički prostor

Faza korisničkog prostora obuhvata period od pokretanja procesa init do trenutka kada aplikacija pošalje prvi UDP datagram sa očitanom temperaturom. U referentnoj konfiguraciji traje 2,925 ± 0,003 s, odnosno 20,7 % ukupnog vremena. U optimizovanoj konfiguraciji svedena je na 94,5 ± 0,3 ms, što predstavlja uštedu od 2,830 ± 0,003 s i smanjenje od 96,8 %, najveće procentualno smanjenje od svih faza. Ta ušteda čini 25,9 % ukupno uštedjenog vremena.

Doprinos pojedinačnih izmena prikazan je u tabeli 9. Vrednosti potiču iz pojedinačnih merenja nad međukonfiguracijama i daju red veličine doprinosa svake mere, dok stavka ostalo obuhvata razliku u odnosu na ukupnu uštedu faze.

Tabela 9: Doprinos izmena korisničkog prostora

Izmena	**Ušteda**
Uspostavljanje mrežne veze	1020 ms
Uklanjanje skripti	680 ms
Uklanjanje podsistema	670 ms
Skraćivanje lanca pokretanja	260 ms
Ostalo	200 ms

Najveća pojedinačna stavka nije bila nijedna skripta, nego čekanje na uspostavljanje veze na mrežnoj sprezi. Ono je u referentnoj konfiguraciji trajalo 1,05 s. Premeštanjem podizanja sprege u konfiguracionu datoteku init programa, čime pregovaranje počinje ranije i teče uporedo sa ostatkom pokretanja, i isključivanjem samog pregovaranja naredbom `ethtool` to vreme je svedeno na oko 30 ms.

Uklanjanjem inicijalizacionih skripti koje ovaj sistem ne koristi uštađeno je 680 ms, a izostavljanjem osamnaest upravljačkih programa multimedijalnog podsistema još 670 ms. Skraćivanjem lanca pokretanja, kojim se aplikacija poziva neposredno iz konfiguracione datoteke init programa umesto preko dva posrednika, uštađeno je dodatnih 260 ms.

5.4 Zbirni pregled

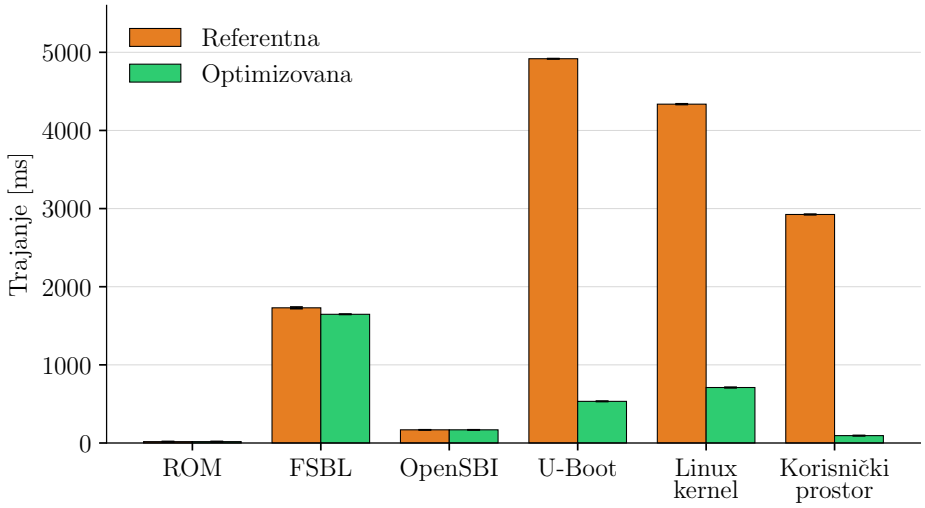
Izmerena vremena po fazama data su u tabeli 10, uz srednju vrednost serije od 50 merenja, standardnu grešku te srednje vrednosti i standardnu devijaciju serije, dok su uštede koje iz njih proizlaze date u tabeli 11. Isti podaci prikazani su i na slici 3, dok slika 4 prikazuje raspodelu ukupnog vremena pokretanja.

Tabela 10: Trajanje faza pokretanja

Faza	**Referentna [ms]**	σ **[ms]**	**Optimizovana [ms]**	σ **[ms]**
ROM	17,53 ± 0,09	0,66	17,69 ± 0,19	1,31
FSBL	1730 ± 10	74	1647,2 ± 0,7	5,0
OpenSBI	168,7 ± 0,3	1,8	168,7 ± 0,3	2,1
U-Boot	4917,9 ± 1,5	10,9	533,0 ± 0,7	5,1
Linux kernel	4336 ± 5	32	710,1 ± 3,3	23,1
Korisnički prostor	2924,6 ± 3,3	23,4	94,5 ± 0,3	2,4
Ukupno	14095 ± 10	73	3171,3 ± 3,2	22,9

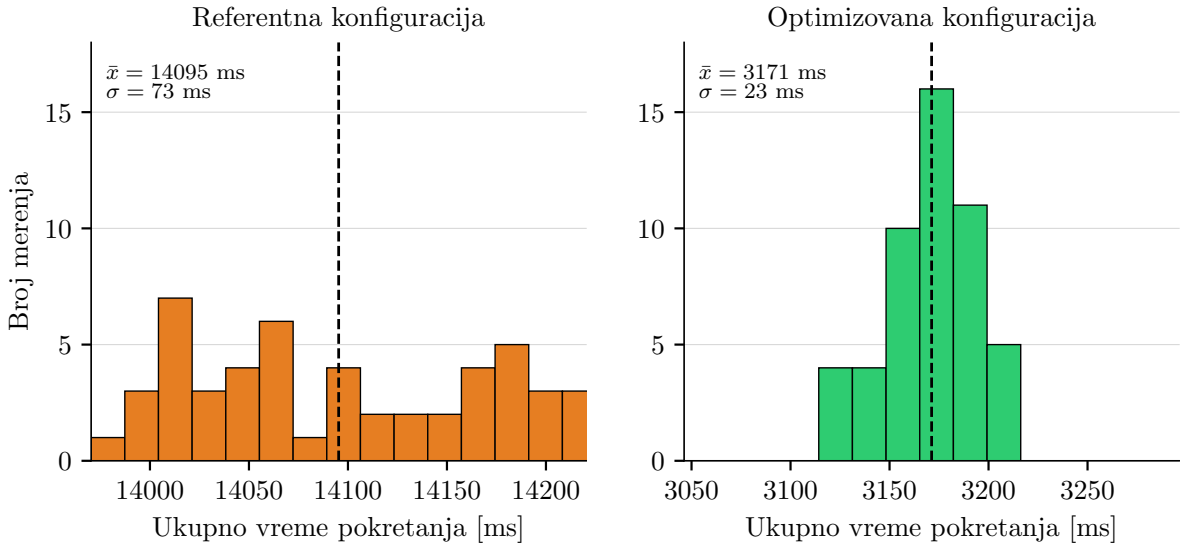
Tabela 11: Uštеđeno vreme po fazama

Faza	**Ušteda [ms]**	**Smanjenje**	**Udeo u uštedi**
ROM	—	—	—
FSBL	83 ± 10	4,8 %	0,8 %
OpenSBI	—	—	—
U-Boot	4384,9 ± 1,7	89,2 %	40,1 %
Linux kernel	3626 ± 6	83,6 %	33,2 %
Korisnički prostor	2830,1 ± 3,3	96,8 %	25,9 %
Ukupno	10924 ± 11	77,5 %	100 %



Slika 3: Trajanje faza pokretanja obe konfiguracije.

Faze ROM i OpenSBI u obe konfiguracije imaju praktično istu izmerenu vrednost, što je očekivano budući da nijedna nije bila predmet izmena, a obe izvršavaju unapred određen niz operacija bez čekanja na spoljašnje događaje. Kod faze OpenSBI srednje vrednosti se razlikuju za manje od 0,1 ms, dok kod faze ROM razlika od 0,2 ms potiče od nekoliko merenja u referentnoj seriji koja odudaraju naniže, pri čemu su medijane obe serije istovetne.



Slika 4: Raspodela ukupnog vremena pokretanja.

Poređenje rasipanja dveju serija pokazuje da optimizacija nije skratila samo srednju vrednost. Standardna devijacija ukupnog vremena pala je sa 73 ms na 23 ms, dakle preko tri puta, pa je pokretanje postalo i znatno predvidljivije u apsolutnom smislu. Relativno rasipanje je pritom blago poraslo, sa 0,52 % na 0,72 %, budući da se ukupno vreme skratilo više nego rasipanje.

Rasipanje nije ravnomerno raspoređeno po fazama. U referentnoj konfiguraciji gotovo celo potiče od faze FSBL, čija merenja obrazuju dve grupe: dvadeset sedam merenja

grupisano je oko 1670 ms, sa standardnom devijacijom od svega 3 ms, dok su preostala rasuta između 1704 i 1869 ms. U optimizovanoj konfiguraciji te pojave nema i standardna devijacija te faze iznosi 5 ms. Uzrok ove pojave nije utvrđen i predstavlja mogući pravac daljeg rada, ali budući da FSBL nije bio predmet izmena, ona ne utiče na skraćenje ostvareno u fazama koje su menjane. Otuda i ušteda od 83 ms koja se za tu fazu javlja u tabeli 11 ne predstavlja rezultat optimizacije, nego posledicu razlike u raspodeli dveju serija.

Histogrami na slici 4 pokazuju i razliku u karakteru rasipanja. Raspodela optimizovane konfiguracije ima izražen vrh oko srednje vrednosti, dok je kod referentne raspoređena gotovo ravnomerno preko celog opsega, što je posledica rasipanja faze FSBL opisanog u prethodnom pasusu.

Najmanje rasipanje pokazuje faza U-Boot u referentnoj konfiguraciji, sa standardnom devijacijom od 10,9 ms, odnosno 0,22 % trajanja te faze. To potvrđuje zapažanje iz odeljka 4.2 da se ta faza svodi na unapred određen niz operacija bez čekanja na spoljašnje događaje.

Zaključak

6.1 Tumačenje rezultata

Rad pokazuje da se vreme pokretanja ugrađenog Linux sistema može višestruko smanjiti bez izmena u primenjenom kodu, bez zamene inicijalizacionog sistema i bez odricanja od funkcionalnosti. Postignuto smanjenje od 89,0 % nad delom sistema koji je bilo moguće menjati nije rezultat jedne odluke nego više izmena, pri čemu nijedna nije zahtevala pisanje novog koda. Sve su se svodile na promene konfiguracije, stabla uređaja ili redosleda operacija.

Ta okolnost sama po sebi predstavlja nalaz. Podrazumevana konfiguracija proizvođača nije bila loša, nego prilagođena drugačijem profilu upotrebe. Kalibracija SD kartice za najbrži režim ima smisla za uređaj koji neprekidno pristupa kartici. Kompresija kernela algoritmom LZMA ima smisla kada je memorija mala i skupa. Učitavanje multimedijalnog podsistema ima smisla na ploči koja radi kao kamera. Nijedna od tih pretpostavki nije važila za sistem koji se jednom pokrene, očita temperaturu i pošalje je preko mreže. Najveći deo posla nije bio u primeni izmena nego u utvrđivanju koje od nasleđenih pretpostavki i dalje važe.

U više slučajeva se pokazalo da je preklapanje delotvornije od uklanjanja. Pregovaranje o mrežnoj vezi nije skraćeno time što je uklonjeno, nego time što je pomereno na početak korisničkog prostora, gde se odvija uporedo sa poslom koji se ionako obavlja. Posao nije nestao, već je smešten u vreme koje je već bilo utrošeno.

Konačno, optimizacija je promenila samu strukturu procesa pokretanja. U referentnoj konfiguraciji faze nedostupne izmeni čine 13,6 % ukupnog vremena, a u optimizovanoj 57,8 %, pri čemu FSBL sam zauzima 51,9 % i predstavlja stavku veću od svih ostalih faza zajedno. Sistem je dostigao tačku u kojoj primenjeni pristup više nema gde da deluje. Dostizanje te granice nije neuspeh nego prirodan ishod postupka i pokazatelj da je prostor dostupan konfiguracijom iscrpljen.

6.2 Buduće izmene

Dalje smanjenje konfiguracije kernela.

Analiza inicijalizacionih poziva pokazala je da posle korekcije za trošak konzole stvaran rad upravljačkih programa iznosi oko 241 ms, pri čemu se većina odnosi na programe koji su neophodni. Ta vrednost je manja od 287,70 ms, koliko u tabeli 8 traje stavka inicijalizacije, budući da zbir trajanja pojedinačnih inicijalizacionih rutina ne obuhvata rad koji se između njih odvija, dok razmak između dva ispisa na konzoli obuhvata sve. Prostor stoga nije u ubrzanju onoga što se izvršava, nego u uklanjanju koda koji se ne izvršava, čime se smanjuje slika kernela i posledično vreme njenog čitanja sa kartice.

Prelazak na fajl sistem bez žurnala.

Korenski fajl sistem je trenutno ext4, čiji žurnal tokom pokretanja zahteva proveru i po potrebi oporavak. U dnevniku referentnog sistema oporavak žurnala javlja se pri svakom pokretanju, a montiranje sa oporavkom troši oko 287 ms. Budući da se korenski sistem tokom rada ne menja, žurnal nema šta da štiti. Prelazak na `SquashFS` uklonio bi tu operaciju u celosti, a kompresija bi istovremeno smanjila broj bajtova koje treba pročitati sa kartice. Opcije `SQUASHFS`, `SQUASHFS_LZ4` i `TMPFS` već su uključene u konfiguraciji jezgra u pripremi za ovu izmenu.

Pokretanje aplikacije kao prvog procesa.

Korisnički prostor je sveden na 94,5 ms, pri čemu se aplikacija pokreće neposredno iz inittaba, posle montiranja fajl sistema i pokretanje mrežnog interfejsa. Krajnji slučaj ovog pristupa bio bi postavljanje argumenta `init=` u komandnoj liniji jezgra tako da pokazuje na skriptu koja obavlja preostale preduslove, pokrene aplikaciju, a zatim naredbom `exec` preda kontrolu stvarnom procesu init.

Time se uklanja sve između predaje kontrole iz jezgra i prvog očitavanja senzora, čime se dobija donja granica onoga što je u korisničkom prostoru uopšte moguće postići. Vrednost ovog merenja nije nužno u tome da se konfiguracija zadrži, nego u tome što pokazuje koliko je od preostalih 94,5 ms posledica same inicijalizacije sistema, a koliko rada koji aplikacija mora obaviti. Ako se te dve vrednosti pokažu bliskim, dalja optimizacija korisničkog prostora nema smisla.

Bibliografija

[1] I. Popović, i Lazar Saranovac, *Namenski računarski sistemi*, 2017.

[2] A. Patra, "RISC-V boot flow: What is next?" in *FOSDEM 2020*, 2020, Pristupljeno: 7. septembra 2026. [Online]. Available: https://archive.fosdem.org/2020/schedule/event/riscv_bootflow/

[3] A. Waterman, K. Asanovic, and J. Hauser, "The RISC-V instruction set manual, volume ii: Privileged architecture," *RISC-V Foundation*, pp. 1–4, 2019.

[4] C. Simmonds, *Mastering Embedded Linux Programming: Harness the power of Linux to create versatile and robust embedded solutions*. Packt Publishing Ltd, 2015.

[5] L. Toikkanen, "Booting operating systems on RISC-V processors," 2025.

[6] Milk-V. Milk-v duos documentation. Pristupljeno: 7. septembra 2026. [Online]. Available: https://milkv.io/docs/duo/getting-started/duos

[7] Milk-V, "duo-buildroot-sdk-v2," 2024, pristupljeno: 7. septembra 2026. [Online]. Available: https://github.com/milkv-duo/duo-buildroot-sdk-v2

[8] B. B. Hall, "Beej's guide to network programming: Using internet sockets," 2026, Pristupljeno: 7. septembra 2026. [Online]. Available: https://beej.us/guide/bgnet/

[9] M. Opdenacker, "Embedded linux boot time optimization," Bootlin training materials, Pristupljeno: 7. septembra 2026. [Online]. Available: https://bootlin.com/doc/training/boot-time/boot-time-slides.pdf

[10] A. Murray, "The right approach to minimal boot times," Embedded Linux Conference, 2010, Pristupljeno: 7. septembra 2026. [Online]. Available: https://elinux.org/images/f/f7/RightApproachMinimalBootTimes.pdf

Firmver mernog mosta

```
import sys
import select
import time
from machine import UART, Pin

uart = UART(0, baudrate=115200, tx=Pin(16), rx=Pin(17))
poll = select.poll()
poll.register(sys.stdin, select.POLLIN)

power_pin = Pin(18, Pin.IN, Pin.PULL_DOWN)

power_on_time = None
first_char_time = None
power_flag = False
char_flag = False
result_printed = False

def power_isr(pin):
    global power_on_time, power_flag, char_flag, result_printed
    if pin.value():  # rising edge: board just powered on
        if not power_flag:
            power_on_time = time.ticks_us()
            power_flag = True
    else:  # falling edge: board just powered off/unplugged
        power_flag = False
        char_flag = False
        result_printed = False

power_pin.irq(trigger=Pin.IRQ_RISING | Pin.IRQ_FALLING, handler=power_isr)

while True:

    if power_flag and char_flag and not result_printed:
        elapsed_us = time.ticks_diff(first_char_time, power_on_time)
        sys.stdout.buffer.write(
            "\r\n[ROM phase] {} us ({:.3f} ms)\r\n".format(
                elapsed_us, elapsed_us / 1000
            ).encode()
        )
        result_printed = True

```

```
42      if uart.any():
43          if power_flag and not char_flag:
44              first_char_time = time.ticks_us()
45              char_flag = True
46          data = uart.read()
47          sys.stdout.buffer.write(data)
48
49      if poll.poll(0):
50          in_data = sys.stdin.buffer.read(1)
51          if in_data:
52              uart.write(in_data)
```

Skripta za obradu rezultata merenja

```
#! /bin/bash

if [ "$1" = "reset" ]; then
    CURR_MEASURE_INDEX=0
fi
CURR_MEASURE_INDEX=${CURR_MEASURE_INDEX:-0}
CURR_MEASURE_INDEX=$((CURR_MEASURE_INDEX + 1))

ROM_PHASE=$(grep -a -i "ROM phase" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/.* \(([0-9]+\.[0-9]+) ms\)
    .*/\1/')
FSBL_START=$(grep -a -i "SCS/" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/')
OPENSBI_START=$(grep -a -i "OpenSBI v0.9" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/')
UBOOT_START=$(grep -a -i "U-Boot 2021.10" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/')
KERNEL_START=$(grep -a -i "Starting kernel ..." /home/vojinv/buildroot-milkv-
    duos/measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/'
    )
USERSPACE_START=$(grep -a -i "Run /sbin/init" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/')
APP_START=$(grep -a -i "Temperature sent" /home/vojinv/buildroot-milkv-duos/
    measurment_results/boot_full.log | sed -E 's/^\[([0-9]+\.[0-9]+).*/\1/')

FSBL_PHASE=$(printf "%.3f" "$(echo "($OPENSBI_START - $FSBL_START) * 1000" | bc)
    ")
OPENSBI_PHASE=$(printf "%.3f" "$(echo "($UBOOT_START - $OPENSBI_START) * 1000" |
     bc)")
UBOOT_PHASE=$(printf "%.3f" "$(echo "($KERNEL_START - $UBOOT_START) * 1000" | bc
    )")
KERNEL_PHASE=$(printf "%.3f" "$(echo "($USERSPACE_START - $KERNEL_START) * 1000"
     | bc)")
USERSPACE_PHASE=$(printf "%.3f" "$(echo "($APP_START - $USERSPACE_START) * 1000"
     | bc)")
TOTAL_TIME=$(printf "%.3f" "$(echo "($APP_START - $FSBL_START) * 1000" | bc)")

echo "
        Measurment number $CURR_MEASURE_INDEX:
                1)ROM Time: $ROM_PHASE ms
                2)FSBL Time: $FSBL_PHASE ms
```

```
                3)OpenSBI Time: $OPENSBI_PHASE ms
                4)U-Boot Time: $UBOOT_PHASE ms
                5)Kernel Time: $KERNEL_PHASE ms
                6)Userspace Time: $USERSPACE_PHASE ms
                7)Total Time: $TOTAL_TIME ms + $ROM_PHASE ms
"  >> ../measurment_results/all_measurments.log
```

Izvorni kod aplikacije

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <fcntl.h>

#define MYPORT 5005


int read_sensor(double *temperature){
        int fd_temperature;
        fd_temperature = open("/sys/bus/iio/devices/iio:device0/in_temp_input",
            O_RDONLY);
        if(fd_temperature < 0){
                perror("open temperature");
                return -1;
                }

        char buf_temp[64];
        int t_read;

        t_read = read(fd_temperature, buf_temp, sizeof buf_temp - 1);

        close(fd_temperature);

        if(t_read <= 0) {
                perror("read");
                return -1;
        }

        buf_temp[t_read] = '\0';

        int temp_value = atoi(buf_temp);

        *temperature = temp_value / 1000.0;

        return 0;
}
```

```
41
42  int main(void){
43          int sockfd;
44          struct sockaddr_in dest_addr;
45          double temperature;
46          char msg[64];
47
48          sockfd = socket(AF_INET, SOCK_DGRAM, 0 );
49          if(sockfd < 0) {
50                  perror("socket");
51                  exit(1);
52          }
53          memset(&dest_addr, 0, sizeof dest_addr);
54          dest_addr.sin_family = AF_INET;
55          dest_addr.sin_port = htons(MYPORT);
56          if (inet_pton(AF_INET, "192.168.10.1", &dest_addr.sin_addr) != 1) {
57                  perror("inet_pton");
58                  return 1;
59          }
60
61          if(read_sensor(&temperature) < 0){
62                  return 1;
63          }
64
65          snprintf(msg, sizeof msg, "temp=%.3f", temperature);
66
67          if (sendto(sockfd, msg, strlen(msg), 0, (struct sockaddr *)&dest_addr,
                sizeof(dest_addr)) < 0){
68                  perror("sendto");
69                  exit(1);
70          }
71          close(sockfd);
72          write(1, "Temperature sent\n",17);
73          return 0;
74
75  }
```

Dodavanje senzora BME280

```
/dts-v1/;
#include "cv181x_base_riscv.dtsi"
#include "cv181x_asic_bga.dtsi"
#include "cv181x_asic_sd.dtsi"
#include "cv181x_default_memmap.dtsi"

/ {
        model = "Milk-V DuoS";
};

&mipi_rx {
        snsr-reset = <&porta 2 GPIO_ACTIVE_LOW>, <&porta 2 GPIO_ACTIVE_LOW>, <&
            porta 2 GPIO_ACTIVE_LOW>;
};

&i2c0 {
        status = "disabled";
};

&i2c1 {
        status = "okay";

        bme280@76 {
                compatible = "bosch,bme280";
                reg = <0x76>;
        };
};

&i2c2 {
        status = "okay";
};
```

Konfiguraciona inittab datoteka

```
# Note: BusyBox init doesn't support runlevels.  The runlevels field is
# completely ignored by BusyBox init. If you want runlevels, use
# sysvinit.
# Format for each entry: <id>:<runlevels>:<action>:<process>
#
# id        == tty to run on, or empty for /dev/console
# runlevels == ignored
# action    == one of sysinit, respawn, askfirst, wait, and once
# process   == program to run
# Startup the system
::sysinit:/bin/echo Run /sbin/init
::sysinit:/bin/mount -t proc proc /proc
::sysinit:/bin/mount -o remount,rw /
::sysinit:/bin/mkdir -p /dev/pts /dev/shm
::sysinit:/bin/mkdir -p /boot
::sysinit:/bin/mount -a
::sysinit:/sbin/ip addr add 192.168.10.2/24 dev eth0
::sysinit:/sbin/ip link set eth0 up
::sysinit:/usr/sbin/ethtool -s eth0 speed 100 duplex full autoneg off
::sysinit:/usr/bin/app
::sysinit:/sbin/swapon -a
null::sysinit:/bin/ln -sf /proc/self/fd /dev/fd
null::sysinit:/bin/ln -sf /proc/self/fd/0 /dev/stdin
null::sysinit:/bin/ln -sf /proc/self/fd/1 /dev/stdout
null::sysinit:/bin/ln -sf /proc/self/fd/2 /dev/stderr
::sysinit:/bin/hostname -F /etc/hostname
# now run any rc scripts
::sysinit:/etc/init.d/rcS
# Put a getty on the serial port
# console::respawn:/sbin/getty -L  console 0 vt100 # GENERIC_SERIAL
console::respawn:/sbin/getty -L  console 115200 vt100 -n -l /usr/local/bin/
    autologin
# Stuff to do for the 3-finger salute
#::ctrlaltdel:/sbin/reboot
# Stuff to do before rebooting
::shutdown:/etc/init.d/rcK
::shutdown:/sbin/swapoff -a
::shutdown:/bin/umount -a -r
```